A9 Testing

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A9 Testing 3
A9-L1 pytest basics, and a SparkSession fixture shared by every test 3
A9-L2 A fast local SparkSession and readable test data 7
A9-L3 Compare DataFrames properly, and test that MERGE is idempotent 11
Track A best-practice checklist 17
Appendix: solutions and answer keys 18

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A9 Testing

Roadmap: Months 16 to 18: MLOps and deployment; Current work. pytest, fixtures, local SparkSession and DataFrame equality checks.

A9-L1 pytest basics, and a SparkSession fixture shared by every test

Concept

pytest finds files named `test_*.py`, runs every function named `test_*`, and reports each failing `assert` with the values on both sides. No classes or special assert methods are needed.

```
def test_weighted_share_uses_weights() -> None:
    assert weighted_share([1, 0], [3.0, 1.0]) == 0.75
```

Fixtures provide what tests need. A test names a fixture as a parameter, and pytest builds it and passes it in. For Spark, one fixture creates the session:

```
@pytest.fixture(scope="session")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    yield session
    session.stop()
```

- `scope="session"` builds it once for the whole test run. Starting Spark takes several seconds. Once per test, and a 40-test suite spends minutes on start-up alone.
- `yield` hands the value to the tests. The code after `yield` runs once at the end, which is where you stop the session.
- Put shared fixtures in `conftest.py`. pytest loads it automatically for every test in that folder and below.

`@pytest.mark.parametrize` runs one test with several inputs, and each input is reported as its own test:

```
@pytest.mark.parametrize(("flags", "weights", "expected"), [([1], [2.0], 1.0), ([0, 0], [1.0, 1.0], 0.0)])
def test_share(flags, weights, expected): ...
```

A good test fails for the bug you fear. Write it against the known answer on small data, including the edge cases: empty input, zero totals, missing values. Check it by breaking the code on purpose and watching the test fail.

Why it matters

A team's test for `weighted_share` checks that the result is between 0 and 1. The function is later rewritten and accidentally returns the unweighted share. The result is still between 0 and 1, so the test passes, and weighted reach in the dashboard is wrong for a month.

A test with a known answer, `weighted_share([1, 0], [3.0, 1.0]) == 0.75`, fails the moment the weights are ignored, because the unweighted answer is 0.5.

Worked example: weak code

```
 1 import pytest
 2 from pyspark.sql import SparkSession
 3
 4
 5 @pytest.fixture
 6 def spark():
 7     session = SparkSession.builder.master("local[2]").getOrCreate()
 8     yield session
 9     session.stop()
10
11
12 def test_share():
13     share = weighted_share([1, 0, 1], [2.0, 1.0, 1.0])
14     assert 0 <= share <= 1
```

1. **Line 5.** Default function scope: Spark starts and stops for every test that uses it.
2. **Line 14.** Passes for almost any wrong implementation. It does not test the rule.

Worked example: strong code

```
 1 from collections.abc import Iterator
 2
 3 import pytest
 4 from pyspark.sql import SparkSession
 5
 6
 7 @pytest.fixture(scope="session")
 8 def spark() -> Iterator[SparkSession]:
 9     """One local session for the whole test run."""
10     session =
   SparkSession.builder.master("local[2]").config("spark.ui.enabled",
   "false").getOrCreate()
12     yield session
13     session.stop()
14
15 @pytest.mark.parametrize(
16     ("flags", "weights", "expected"),
```

```
    [
        ([1, 0, 1], [2.0, 1.0, 1.0], 0.75),
        ([0, 0], [1.0, 1.0], 0.0),
        ([1, 0], [0.0, 0.0], None),
    ],
)
def test_weighted_share(flags: list[int], weights: list[float], expected:
    float | None) -> None:
    assert weighted_share(flags, weights) == expected
```

1. **Line 7.** One Spark start-up for the whole run.
2. **Line 11.** Tests run here. The line after yield is the teardown.
3. **Line 18.** A known answer that differs from the unweighted one (0.67), so ignoring weights fails.
4. **Line 20.** The edge case: zero total weight must give None.

One session-scoped fixture, and tests with known answers that fail for the bugs you care about.

Recap card

- **What does pytest run?** Functions named test_* in files named test_*.py. Plain assert statements.
- **SparkSession fixture scope** scope="session": start Spark once per test run.
- **yield in a fixture** Hands the value to tests. Code after yield is the teardown.
- **What makes a test worth having?** It fails for the bug you fear: known answers that separate right from wrong, plus edge cases.

Practice

Exercise 1. Multiple choice (level 1)

Which fixture scope suits a SparkSession used by 40 tests?

- a) function
- b) session
- c) No fixture: create a session inside each test
- d) class

Why?

- r1) Spark takes seconds to start, and tests can safely share one local session.
- r2) Only session-scoped fixtures can use yield.

Exercise 2. Predict the output (level 2)

How many tests does pytest report for this file?

```python
import pytest


@pytest.mark.parametrize("wave", ["W13", "W14", "W15"])
def test_wave_loads(wave: str) -> None:
    assert wave.startswith("W")


def test_empty_wave_list() -> None:
    assert unprocessed([]) == []
```

Exercise 3. Fill the gap (level 2)

Complete the fixture so Spark starts once per test run and stops at the end.

```python
@pytest._____(1)(scope="_____(2)")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    _____(3) session
    session.stop()
```

Exercise 4. Write the function (level 4)

`weighted_share(flags, weights)` returns the share of weight where the flag is 1, or `None` when the total weight is 0. Write **the test**, `test_weighted_share()`, not the function.

The checker runs your test three times: against a correct implementation (it must pass), against one that ignores the weights, and against one that returns 0.0 instead of None for a zero total (both must fail).

Your code must pass these checks:

- Your test fails against a correct implementation. Check the expected values.
- Your test passes when the weights are ignored. Use weights where the weighted and unweighted answers differ, such as [1, 0] with [3.0, 1.0].
- Your test passes when a zero total returns 0.0. Add a case with all weights 0 and assert the result `is None`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
def test_weighted_share() -> None: ...
```

Starter code:

```python
def test_weighted_share() -> None:
    ...
```

Exercise 5. Spot the bug (level 5)

The test suite takes six minutes for 45 small tests. Tap the line that causes most of it, then fix the fixture.

```
1 import pytest
2 from pyspark.sql import SparkSession
3
4
5 @pytest.fixture
6 def spark():
7     session = SparkSession.builder.master("local[2]").getOrCreate()
8     yield session
9     session.stop()
```

Write down the line number of the bug. Then write the fix.

Sources

- pytest collects one test per parametrize value; fixtures with yield run teardown after; scope='session' builds once per run. (verified: pytest 9.1.1 behaviour, used by this repository's own test suite.)

A9-L2 A fast local SparkSession and readable test data

Concept

Unit tests for PySpark run on your laptop and in CI, not on a cluster. A local session that starts in seconds and runs small jobs fast needs a few settings:

```
1 builder = (
2     SparkSession.builder.master("local[2]")
3     .config("spark.sql.shuffle.partitions", "2")
4     .config("spark.ui.enabled", "false")
5     .config("spark.sql.extensions",
    "io.delta.sql.DeltaSparkSessionExtension")
6     .config("spark.sql.catalog.spark_catalog",
    "org.apache.spark.sql.delta.catalog.DeltaCatalog")
7     .config("spark.sql.sources.default", "delta")
8 )
9 spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

- `local[2]` runs Spark in one process with two worker threads. Two threads, not one, so bugs that depend on partitioning still show up.
- `spark.sql.shuffle.partitions` defaults to 200. On ten test rows, every shuffle creates 200 tiny tasks. Two is plenty.
- `configure_spark_with_delta_pip` from `delta-spark` adds the matching Delta JAR. The two `config` lines enable Delta SQL and the Delta catalog.

- Open-source Spark writes `saveAsTable` as Parquet by default. Databricks uses Delta. Set the default so tests behave like production.

Test data. Build it with an explicit schema, as a DDL string such as `"respondent_id STRING, final_weight DOUBLE"`. Inference guesses types from values, fails on a column of only `None` (`CANNOT_DETERMINE_TYPE`), and turns `2` into a LONG where production has DOUBLE.

A **test data builder** keeps tests short: one function returns a valid row with sensible defaults, and each test overrides only the field it is about.

```
1 def respondent(**overrides: object) -> dict[str, object]:
2     row = {"respondent_id": "R1", "wave_id": "W1", "final_weight": 1.0,
    "in_scope": True}
3     row.update(overrides)
4     return row
```

A test that says `respondent(final_weight=None)` tells the reader exactly what it is testing.

Why it matters

A project's Spark tests take 14 minutes in CI, so developers stop running them locally. The session uses the default 200 shuffle partitions, and each of the 60 tests runs two or three shuffles on a handful of rows. Most of the time goes into scheduling empty tasks. A MERGE bug that a test would have caught ships to production, because nobody waited for the suite.

With `local[2]` and two shuffle partitions, the same suite runs in under two minutes, and people run it before every push.

Worked example: weak code

```
1 @pytest.fixture(scope="session")
2 def spark():
(1)     return SparkSession.builder.getOrCreate()
4
5
6 def test_null_weights_are_reported(spark):
(2)     df = spark.createDataFrame([{"respondent_id": "R1", "final_weight":
    None, "wave_id": "W1", "in_scope": True}])
8     assert null_weight_count(df) == 1
```

1. **Line 3.** No master, 200 shuffle partitions, UI on, no Delta: slow and unlike production.
2. **Line 7.** Inferred schema: a column of only None raises CANNOT_DETERMINE_TYPE, and every field is spelt out in every test.

Worked example: strong code

```
1  RESPONDENT_SCHEMA = "respondent_id STRING, wave_id STRING, final_weight DOUBLE, in_scope BOOLEAN"
2
3
4  def respondent(**overrides: object) -> dict[str, object]:
5      """A valid silver respondent. Override only the fields a test is about."""
6      row: dict[str, object] = {"respondent_id": "R1", "wave_id": "W1", "final_weight": 1.0, "in_scope": True}
7      unknown = set(overrides) - set(row)
8      if unknown:
2          raise KeyError(f"unknown fields: {sorted(unknown)}")
10     row.update(overrides)
11     return row
12
13
14 def respondents_df(spark: SparkSession, rows: list[dict[str, object]]) -> DataFrame:
15     """DataFrame with the silver schema, whatever the values."""
16     fields = [f.split()[0] for f in RESPONDENT_SCHEMA.split(", ")]
17     return spark.createDataFrame([tuple(r[f] for f in fields) for r in rows], RESPONDENT_SCHEMA)
18
19
20 def test_null_weights_are_reported(spark: SparkSession) -> None:
3      df = respondents_df(spark, [respondent(), respondent(respondent_id="R2", final_weight=None)])
22     assert null_weight_count(df) == 1
```

1. **Line 1.** One DDL schema: types match production, and None is always allowed.
2. **Line 9.** A typo such as final_wieght fails loudly instead of silently adding nothing.
3. **Line 21.** The test reads as its intent: a second respondent with a missing weight.

A tuned local session makes tests fast enough to run on every change. Builders with explicit schemas make them short and exact.

Recap card

- **Local test session settings** master("local[2]"), shuffle partitions 2, UI off, Delta configured with configure_spark_with_delta_pip.
- **Default spark.sql.shuffle.partitions** 200: far too many for test data.
- **Why explicit schemas in test data?** Inference fails on all-None columns and can pick different types from production.
- **Test data builder** A function with valid defaults. Tests override only the fields they are about, and typos raise.

Practice

Exercise 1. Predict the output (level 1)

A fresh local session on PySpark 4.0 with no extra configuration. What does this print?

```
print(spark.conf.get("spark.sql.shuffle.partitions"))
```

Exercise 2. Fill the gap (level 2)

Complete the local test session: two threads, two shuffle partitions, with the Delta JAR added.

```
builder = (
    SparkSession.builder.master("_____(1)")
    .config("spark.sql.shuffle.partitions", "_____(2)")
    .config("spark.sql.extensions",
    "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
    "org.apache.spark.sql.delta.catalog.DeltaCatalog")
)
spark = _____(3)(builder).getOrCreate()
```

Exercise 3. Write the function (level 3)

Write `respondent(**overrides)`. It returns a dict with defaults `respondent_id="R1"`, `wave_id="W1"`, `final_weight=1.0`, `in_scope=True`, updated with the overrides. Raise `KeyError` naming any override that is not one of those four fields. Each call must return a new dict.

Your code must pass these checks:

- With no overrides, return the four defaults.
- Overrides replace only the named fields.
- An unknown field must raise KeyError naming it, or typos slip through.
- Each call must return a fresh dict. Changing one must not change the defaults.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# A valid silver respondent row. Override only the fields a test is about.
def respondent(**overrides: object) -> dict[str, object]: ...
```

Starter code:

```
def respondent(**overrides: object) -> dict[str, object]:
    ...
```

Exercise 4. Spot the bug (level 4)

This helper builds test data. It works for most tests, but fails with `CANNOT_DETERMINE_TYPE` in the test for missing weights. Tap the line, then fix the function with an explicit schema.

```
1 def weights_df(spark: SparkSession, weights: list[float | None]) ->
    DataFrame:
2     """One respondent per weight."""
3     rows = [(f"R{i}", w) for i, w in enumerate(weights)]
4     return spark.createDataFrame(rows, ["respondent_id", "final_weight"])
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Put in order (level 5)

Order the set-up of a local PySpark test harness.

- Write a session-scoped spark fixture in conftest.py
- Create a virtual environment and install pyspark, delta-spark and pytest
- Write tests with known answers against the builders
- Write test data builders with explicit schemas
- Install Java 17 or 21

Sources

- spark.sql.shuffle.partitions defaults to 200; createDataFrame with only column names infers types and fails on all-None columns. (verified: Run on PySpark 4.0.4 in this repository.)
- configure_spark_with_delta_pip adds the Delta package matching the installed delta-spark version. (verified: Used by practice/conftest.py with delta-spark 4.0.1.)

A9-L3 Compare DataFrames properly, and test that MERGE is idempotent

Concept

Comparing DataFrames by hand is harder than it looks. `actual.collect() == expected.collect()` fails when the rows come back in a different order, which Spark never guarantees. It also fails when a float is `0.30000000000000004` instead of `0.3`. Sorting both sides fixes the order but not the floats, and neither checks the schema.

`pyspark.testing` (PySpark 3.5 and later) handles all of this:

```
1 from pyspark.testing import assertDataFrameEqual, assertSchemaEqual
```

```
2
3 assertDataFrameEqual(actual, expected)
```

Tested on PySpark 4.0.4:

- Row order is ignored by default. Pass `checkRowOrder=True` when order is the point, as in a top-10 list.
- Floats compare with a tolerance (`rtol=1e-5`, `atol=1e-8` by default).
- Schemas must match: INT against BIGINT fails with `DIFFERENT_SCHEMA`.
- Duplicates count: two identical rows do not equal one.
- `expected` can be a list of `Row` objects, which is handy for snapshots.
- Failures raise `PySparkAssertionError`, a subclass of `AssertionError`, so pytest reports them as ordinary test failures, with a diff.

`assertSchemaEqual(actual.schema, expected.schema)` checks only the structure.

Testing MERGE. The property that matters is idempotence. The test:

1. Create an empty Delta table with the target schema.
2. Run the load once, and take a snapshot: `first = spark.table(t).collect()`.
3. Run the same load again.
4. `assertDataFrameEqual(spark.table(t), first)`.

Add cases for the failures you fear: a repeated key in the source, a redelivery with a newer version, and the same respondent in two waves. Each one takes a few rows and a second or two.

In the practice harness, PySpark 4.0.4 needs pandas 2.x. With pandas 3.x, `assertDataFrameEqual` fails to import.

Why it matters

A silver test compares `actual.collect() == expected.collect()`. It passes on a developer's laptop and fails one CI run in five, because the order of rows varies with partitioning. The team marks it flaky and skips it. A month later a real regression in the MERGE, duplicate rows after a retry, ships untested.

`assertDataFrameEqual` ignores order, so the test is stable, and an idempotence test that runs the load twice catches the regression.

Worked example: weak code

```
1 def test_silver_load(spark):
1     upsert_silver(spark, source, "silver", ["respondent_id", "wave_id"],
  "delivery_version")
```

```
❷     assert spark.table("silver").collect() == expected.collect()
```

1. **Line 2.** One run only. It never checks what a retry does.
2. **Line 3.** Order-dependent and exact on floats: flaky, and a failure shows no useful diff.

Worked example: strong code

```
 1 from pyspark.testing import assertDataFrameEqual
 2
 3 KEYS = ["respondent_id", "wave_id"]
 4 SCHEMA = "respondent_id STRING, wave_id STRING, final_weight DOUBLE,
     delivery_version INT"
 5
 6
 7 def test_silver_load_is_idempotent(spark: SparkSession, empty_silver:
     str) -> None:
 8     source = spark.createDataFrame(
❶          [("R1", "W1", 1.2, 1), ("R1", "W1", 1.5, 2), ("R2", "W1", 0.8,
     1)], SCHEMA
10     )
11     upsert_silver(spark, source, empty_silver, KEYS, "delivery_version")
❷     first = spark.table(empty_silver).collect()
13
14     upsert_silver(spark, source, empty_silver, KEYS, "delivery_version")
15
❸     assertDataFrameEqual(spark.table(empty_silver), first)
17     expected = spark.createDataFrame([("R1", "W1", 1.5, 2), ("R2", "W1",
     0.8, 1)], SCHEMA)
❹     assertDataFrameEqual(spark.table(empty_silver), expected)
```

1. **Line 9.** The source includes a repeated key, the case that breaks a naive MERGE.
2. **Line 12.** A snapshot of the table after one run, as a list of Rows.
3. **Line 16.** Second run leaves the table unchanged: idempotence, order-independent.
4. **Line 18.** And the content is right: the newest version of R1 won.

One helper compares DataFrames the right way, and one small test proves a MERGE survives a retry and a repeated key.

Recap card

- **Compare two DataFrames in a test**
  pyspark.testing.assertDataFrameEqual(actual, expected). Ignores order, tolerates float rounding, checks schema.
- **When should row order matter?** Only for ranked output. Pass checkRowOrder=True.

- **Idempotence test for a MERGE** Load, snapshot with collect(), load again, assertDataFrameEqual(table, snapshot).
- **Why collect() for the snapshot?** A DataFrame is lazy and would re-read the table after the second run.

Practice

Exercise 1. Multiple choice (level 1)

`actual` holds rows (1, 0.3) and (2, 1.0). `expected` holds (2, 1.0) and (1, 0.1 + 0.2). Both have the schema `id INT, w DOUBLE`. What does `assertDataFrameEqual(actual, expected)` do on PySpark 4.0?

- a) Passes
- b) Fails because the rows are in a different order
- c) Fails because 0.1 + 0.2 is not exactly 0.3
- d) Fails because it needs checkRowOrder=True

Why?

- r1) It ignores row order by default and compares floats with a relative tolerance.
- r2) It only compares schemas.

Exercise 2. Predict the output (level 2)

A hand-written comparison sorts both sides first. What does it print?

```
actual = [(1, 0.1 + 0.2), (2, 1.0)]
expected = [(2, 1.0), (1, 0.3)]
print(sorted(actual) == sorted(expected))
```

Exercise 3. Fill the gap (level 2)

Import the helpers, then check that the top-10 regions come back in the right order.

```
from pyspark._____(1) import assertDataFrameEqual, assertSchemaEqual

assertDataFrameEqual(top_regions, expected, _____(2)=True)
```

Exercise 4. Write the function (level 4)

Write `check_idempotent(spark, load, table)`. `load` is a function that takes no arguments and loads a fixed source into `table`. Run it once, snapshot the table with `collect()`, run it again, and assert the table still equals the snapshot with `assertDataFrameEqual`.

What you need to know.

`Callable` from `collections.abc` types a parameter that holds a function. `Callable[[], None]` is a function that takes no arguments and returns nothing. You run it by calling the parameter, like any function:

```
from collections.abc import Callable

def twice(step: Callable[[], None]) -> None:
    step()
    step()
```

`.collect()` returns a list of rows, a snapshot that does not change when the table changes later.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Assert that running load twice leaves table as one run did.
def check_idempotent(
    spark: SparkSession,
    load: Callable[[], None],
    table: str,
) -> None: ...
```

Starter code:

```
from collections.abc import Callable

from pyspark.sql import SparkSession
from pyspark.testing import assertDataFrameEqual


def check_idempotent(spark: SparkSession, load: Callable[[], None],
   table: str) -> None:
     ...
```

Exercise 5. Spot the bug (level 5)

This test fails about one CI run in five, though the code under test is correct. Tap the line, then fix the test.

```
def test_gold_metrics(spark: SparkSession) -> None:
    silver = spark.createDataFrame([("R1", "W1", "N", 1.0), ("R2", "W1",
   "S", 1.0)], "respondent_id STRING, wave_id STRING, region STRING,
   final_weight DOUBLE")
    expected = spark.createDataFrame([("N", 1.0), ("S", 1.0)], "region
   STRING, weighted_base DOUBLE")
    actual = aggregate_base(silver).select("region", "weighted_base")
    assert actual.collect() == expected.collect()
```

Write down the line number of the bug. Then write the fix.

Sources

- assertDataFrameEqual ignores row order by default, uses rtol=1e-5 and atol=1e-8, fails on schema differences and duplicates, accepts a list of Rows as expected, and raises PySparkAssertionError (a subclass of AssertionError). (verified: Run on PySpark 4.0.4 with pandas 2.3.3 in this repository.)
- PySpark 4.0.4 fails to import assertDataFrameEqual with pandas 3.x. (verified: ImportError: cannot import name '_builtin_table' from 'pandas.core.common', reproduced with pandas 3.0.6.)

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A9-L1 pytest basics, and a SparkSession fixture shared by every test

A9-L1 exercise 1

Hints. (1) How long does a SparkSession take to start? (2) How often should it start in one test run? (3) Once.

Answer. b) session Because: r1) Spark takes seconds to start, and tests can safely share one local session.

`scope="session"` starts Spark once for the whole run. Tests stay independent as long as each creates its own data and tables.

Common wrong answers.

- `a`: Function scope rebuilds the fixture for every test: 40 Spark start-ups.
- `c`: Same cost as function scope, plus duplicated set-up code in every test.
- `d`: Class scope rebuilds per test class. Session scope builds once.
- `b|r2`: Right scope, wrong reason. Fixtures of any scope can yield. Session scope wins on start-up cost.

A9-L1 exercise 2

Hints. (1) How many values does parametrize receive? (2) Each value becomes its own test. (3) 3 + 1.

Answer.

```
4
```

`parametrize` generates one test per value, so pytest reports `test_wave_loads[W13]`, `[W14]`, `[W15]` and `test_empty_wave_list`: four.

Common wrong answers.

- `2`: Each parametrize value is a separate test: three for test_wave_loads, plus one.
- `3`: Three from the parametrized test, plus test_empty_wave_list: four.

A9-L1 exercise 3

Hints. (1) The decorator that declares a fixture. (2) The widest scope. (3) The keyword that pauses a function and resumes it later.

Answer.

```
@pytest.fixture(scope="session")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    yield session
    session.stop()
```

`@pytest.fixture(scope="session")` builds the fixture once. `yield` gives the session to the tests, and the line after it runs at the end.

Common wrong answers.

- g3 = `return`: `return` ends the function, so `session.stop()` never runs. `yield` hands over the session and resumes for teardown.
- g2 = `module`: Module scope starts Spark once per test file. Session scope starts it once for the run.

A9-L1 exercise 4

Hints. (1) Pick inputs where the weighted and unweighted answers differ. (2) [1, 0] with weights [3.0, 1.0]: weighted 0.75, unweighted 0.5. (3) Add `assert weighted_share([1, 0], [0.0, 0.0]) is None`.

Answer.

```
def test_weighted_share() -> None:
    assert weighted_share([1, 0], [3.0, 1.0]) == 0.75
    assert weighted_share([1, 0], [0.0, 0.0]) is None
```

A test proves something only if a plausible bug makes it fail. Known answers chosen to separate the right rule from the wrong one, plus the edge case, catch both bugs.

A9-L1 exercise 5

Hints. (1) What is the default fixture scope? (2) How many times does Spark start in 45 tests? (3) `@pytest.fixture(scope="session")`.

Answer. The bug is on line 5.

Fixed version:

```
import pytest
from pyspark.sql import SparkSession


@pytest.fixture(scope="session")
```

```
6 def spark():
7     session = SparkSession.builder.master("local[2]").getOrCreate()
8     yield session
9     session.stop()
```

The default function scope starts and stops Spark for every test, at several seconds each. Session scope starts it once.

Common wrong answers.

- Line 9: Stopping the session in teardown is right. The problem is how often the fixture runs.
- Line 7: The builder is fine. How many times is it called in the run?

A9-L2 A fast local SparkSession and readable test data

A9-L2 exercise 1

Hints. (1) The default suits clusters, not tests. (2) It is a round number in the hundreds. (3) 200.

Answer.

```
1 200
```

The default is 200, which suits large data on a cluster. On test data it creates hundreds of empty tasks per shuffle. Set it to 2.

Common wrong answers.

- `2`: Two is what you should set for tests. The default is 200.
- `auto`: Databricks can manage shuffle partitions automatically, but open-source Spark's default setting is 200.

A9-L2 exercise 2

Hints. (1) The local master takes the thread count in brackets. (2) Match the shuffle partitions to the threads. (3) The helper from the delta package ends with `_pip`.

Answer.

```
1 builder = (
2     SparkSession.builder.master("local[2]")
3     .config("spark.sql.shuffle.partitions", "2")
4     .config("spark.sql.extensions",
    "io.delta.sql.DeltaSparkSessionExtension")
5     .config("spark.sql.catalog.spark_catalog",
    "org.apache.spark.sql.delta.catalog.DeltaCatalog")
6 )
7 spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

`local[2]` runs two threads in-process, two shuffle partitions avoid empty tasks, and `configure_spark_with_delta_pip` adds the Delta JAR that matches your delta-spark version.

Common wrong answers.

- g1 = `local[1]`: One thread hides bugs that only appear with several partitions. Use two.
- g1 = `yarn`: yarn needs a cluster. Unit tests run in-process with `local[...]`.
- g3 = `configure_delta`: The delta-spark helper is `configure_spark_with_delta_pip`.

A9-L2 exercise 3

Hints. (1) Build the defaults dict inside the function. (2) Compare the override keys with the default keys. (3) Raise KeyError for unknown keys, then update and return.

Answer.

```
1  def respondent(**overrides: object) -> dict[str, object]:
2      """A valid silver respondent row. Override only the fields a test is
       about.
3
4      Raises:
5          KeyError: If an override names an unknown field.
6      """
7      row: dict[str, object] = {"respondent_id": "R1", "wave_id": "W1",
       "final_weight": 1.0, "in_scope": True}
8      unknown = set(overrides) - set(row)
9      if unknown:
10         raise KeyError(f"unknown fields: {sorted(unknown)}")
11     row.update(overrides)
12     return row
```

A fresh dict per call keeps tests independent. Rejecting unknown fields turns a typo in a test into an immediate error instead of a test that checks nothing.

A9-L2 exercise 4

Hints. (1) Column names alone leave Spark to infer types. (2) What can it infer from [None, None]? (3) Pass a DDL string with the types.

Answer. The bug is on line 4.

Fixed version:

```
1  def weights_df(spark: SparkSession, weights: list[float | None]) ->
       DataFrame:
2      """One respondent per weight."""
3      rows = [(f"R{i}", w) for i, w in enumerate(weights)]
```

```
4     return spark.createDataFrame(rows,
      "respondent_id STRING, final_weight DOUBLE")
```

A list of names gives Spark no types, so it infers them. With only `None` weights there is nothing to infer from. A DDL schema string fixes the types for every test.

Common wrong answers.

- Line 3: Building tuples is fine. The problem is that Spark has to guess the types.

A9-L2 exercise 5

Hints. (1) Spark runs on the JVM. (2) Tests depend on the fixture and the builders. (3) Infrastructure first, then data helpers, then tests.

Answer.

1. Install Java 17 or 21
2. Create a virtual environment and install pyspark, delta-spark and pytest
3. Write a session-scoped spark fixture in conftest.py
4. Write test data builders with explicit schemas
5. Write tests with known answers against the builders

Java, then the Python environment, then the shared fixture, then data builders, then the tests that use them.

Common wrong answers.

- Either order installs fine, but Spark needs Java before anything runs: check it first so the first test run does not fail.
- Tests need the spark fixture to create DataFrames. Write it before the tests.

A9-L3 Compare DataFrames properly, and test that MERGE is idempotent

A9-L3 exercise 1

Hints. (1) What is the default for checkRowOrder? (2) How are floats compared? (3) Both differences are tolerated.

Answer. a) Passes Because: r1) It ignores row order by default and compares floats with a relative tolerance.

`assertDataFrameEqual` ignores row order unless you set `checkRowOrder=True`, and compares floats within `rtol=1e-5`. Tested on PySpark 4.0.4.

Common wrong answers.

- `b`: checkRowOrder is False by default, so order is ignored.
- `c`: Floats compare with rtol=1e-5 by default, so 0.30000000000000004 equals 0.3.
- `d`: checkRowOrder=True would make it stricter. The default ignores order.
- `a|r2`: Right outcome, wrong reason. It compares rows too. assertSchemaEqual compares only schemas.

A9-L3 exercise 2

Hints. (1) Sorting handles the order. (2) What is 0.1 + 0.2 exactly? (3) Exact equality on floats.

Answer.

```
False
```

Sorting fixes the order but not floating-point rounding. That is why a helper with tolerances beats hand-written comparisons.

Common wrong answers.

- `True`: Sorting fixes the order, but 0.1 + 0.2 is 0.30000000000000004, so the tuples differ.

A9-L3 exercise 3

Hints. (1) The module name says what it is for. (2) The parameter name says what it checks. (3) `testing` and `checkRowOrder`.

Answer.

```
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual

assertDataFrameEqual(top_regions, expected, checkRowOrder=True)
```

`pyspark.testing` provides the helpers. `checkRowOrder=True` makes order part of the comparison, which matters for ranked output.

Common wrong answers.

- g1 = `sql`: The test helpers live in `pyspark.testing`.
- g2 = `ordered`: The parameter is `checkRowOrder`.

A9-L3 exercise 4

Hints. (1) load(), snapshot, load() again. (2) The snapshot must be materialised with collect(), or it re-reads the changed table. (3) `assertDataFrameEqual(spark.table(table), first)`.

Answer.

```python
from collections.abc import Callable

from pyspark.sql import SparkSession
from pyspark.testing import assertDataFrameEqual


def check_idempotent(spark: SparkSession, load: Callable[[], None],
    table: str) -> None:
    """Assert that running load twice leaves table as one run did.

    Raises:
        AssertionError: If the second run changed the table.
    """
    load()
    first = spark.table(table).collect()
    load()
    assertDataFrameEqual(spark.table(table), first)
```

`collect()` freezes the first state. A DataFrame would be lazy and re-read the table after the second run, so the comparison would always pass. `assertDataFrameEqual` then compares without caring about order.

A9-L3 exercise 5

Hints. (1) Does a groupBy return rows in a fixed order? (2) collect() lists keep that order. (3) Use assertDataFrameEqual, which ignores order.

Answer. The bug is on line 5.

Fixed version:

```python
def test_gold_metrics(spark: SparkSession) -> None:
    silver = spark.createDataFrame([("R1", "W1", "N", 1.0), ("R2", "W1",
    "S", 1.0)], "respondent_id STRING, wave_id STRING, region STRING,
    final_weight DOUBLE")
    expected = spark.createDataFrame([("N", 1.0), ("S", 1.0)], "region
    STRING, weighted_base DOUBLE")
    actual = aggregate_base(silver).select("region", "weighted_base")
    assertDataFrameEqual(actual, expected)
```

Aggregation output has no guaranteed order, so the two lists match only when the partitions happen to line up. `assertDataFrameEqual` compares the rows as a multiset.

Common wrong answers.

- Line 4: Selecting the columns to compare is fine. The flakiness comes from how they are compared.